Universidad Tecnológica Nacional — Tecnicatura Universitaria en Programación
Base de Datos II

Informe Técnico — Segunda Entrega

Trabajo Práctico Integrador · Unidades 4 a 8
Food Store

INTEGRANTES

Pamela Chirino
Lucas Agüero
Mayra Mulé

Mendoza, 9 de noviembre de 2026

Informe Técnico — Segunda Entrega del TPI (Unidades 4 a 8)

Proyecto: Food Store · Base de Datos II · UTN · **Integrantes:** Pamela Chirino, Lucas Agüero, Mayra Mulé

Este informe resume lo implementado en las Unidades 4 a 8, consolidando los informes individuales de cada unidad (`informes/TP_U4_FNBC_*`, `docs/informe_auditoria.md`, `docs/informe_incidente.md`, `informes/informe_u6.pdf`, `informes/informe_u8.pdf`, y el TP de NoSQL en el repositorio separado `nosql-mongo`), según pide el punto "Informe técnico a acompañar" de las condiciones de entrega.

1. Qué se implementó en cada unidad

Unidad 4 — Normalización avanzada y redundancia. Se refactorizó `control_lote_almacen` (violaba FNBC por `responsable_control_id → deposito_id`) en dos tablas normalizadas más una vista de compatibilidad (`v_control_lote_almacen_fnbc`), migrando los datos existentes sin pérdida (`scripts/tp_fnbc_control_lote.sql`, especificado en `specs/spec_migracion_fnbc.md`). Por el lado de la desnormalización controlada, se agregó `mv_top_categorias_dia` (vista materializada con índice único) para bajar el costo de un panel de reportes de ~1286 a ~37 de costo estimado, documentando explícitamente que es reversible sin pérdida (`DROP MATERIALIZED VIEW` alcanza).

Unidad 5 — Administración y seguridad. Roles de mínimo privilegio sobre las tablas reales (`sql/roles.sql`), auditoría activada a nivel de servidor (`postgresql.conf`, `log_statement='mod'`) y a nivel de aplicación (`log_auditoria` en JSONB), simulacro de incidente real con intentos de login fallidos, lectura de `usuario` y un intento de escalamiento de privilegios bloqueado por el propio esquema de permisos (`sql/simulacro_incidente.sql`, `docs/informe_incidente.md`), y auditoría del esquema de permisos asistida por IA con una propuesta rechazada explícitamente (ver sección 4).

Unidad 6 — Mantenimiento. Plan de mantenimiento completo (`specs/SPEC_U6.md`): respaldo diario automatizado, chequeo de salud, roles `mant_backup` / `mant_monitor` de mínimo privilegio, y un simulacro de desastre real (`scripts/restore_test.sh`) con restauración completa + validación de integridad.

Unidad 7 — NoSQL. Migración de 5 pedidos reales de Food Store a una colección documental en MongoDB (cliente y líneas embebidos), con las 4 consultas pedidas y sus 2 equivalentes en PostgreSQL verificadas con IDs idénticos, y contraste empírico de 2 afirmaciones típicas sobre Mongo vs. PostgreSQL. **Nota de repositorio:** este TP se desarrolló en un repositorio Git separado (`nosql-mongo`), no fusionado a `food-store`, por tratarse de un motor y una entrega independientes del proyecto integrador; el detalle completo está en `informe_u7.pdf` de ese repositorio.

Unidad 8 — Bases de datos móviles e integración. Base SQLite de la tablet (`movil/schema_sqlite.sql`) con su esquema y su sincronización especificados en Kiro (`docs/kiro_tp8/`) y generados con OpenCode, sincronización simulada contra el servidor vía snapshot + semáforo (`movil/semaforo.sql`), con 3 casos de conflicto probados (cliente acepta con UUID nuevo, catálogo sin el producto, reenvío duplicado rechazado), y code review final con 6 hallazgos documentados (`docs/evidencia/verificaciones_code_review.txt`), incluida la decisión humana sobre cada uno.

2. Cómo se probó cada parte

- **U4:** verificación de equivalencia con `EXCEPT` cruzado entre el esquema legacy y la vista de compatibilidad (0 filas en ambas direcciones) y chequeo de la DF1 sobre el esquema nuevo.
- **U5:** `sql/roles.sql` se probó en PostgreSQL real verificando idempotencia (dos corridas sin error); el simulacro de incidente se ejecutó contra una base real y se contrastó la reconstrucción de la IA contra la evidencia real del log; la anonimización de `usuario_anon` se verificó con `EXCEPT` (0 filas de diferencia en

los agregados).
- **U6:** ciclo completo de respaldo + restauración + validación corrido varias veces contra PostgreSQL real, con volumen representativo (~115.000 filas).
- **U7:** las 4 consultas MongoDB y sus 2 equivalentes PostgreSQL se corrieron contra datos reales migrados desde Food Store, no sobre datos inventados; los 2 contrastes de afirmaciones de IA se midieron empíricamente (inserción con FK inexistente, medición de latencia con conexión ya abierta).
- **U8:** las pruebas SQLite (`movil/pruebas_sqlite.sql`) y los 4 casos de conflicto se corrieron contra una base SQLite real; el code review final se verificó ejecutando cada hallazgo contra la base real, no por lectura del código.

3. Qué resultados se obtuvieron

- **RTO observado (simulacro de desastre, U6):** 0,71 segundos (contra un objetivo previsto de 1 hora).
- **RPO observado (U6):** 2 segundos de antigüedad del respaldo al momento del simulacro (contra un objetivo previsto de 5 minutos).
- **Informe post-incidente (U5):** el intento de escalamiento de privilegios falló como estaba previsto por el diseño de roles; no se encontró una vulnerabilidad real, pero sí un error en una versión anterior del propio script de simulacro, que afirmaba un resultado que el esquema de permisos real no permite — corregido antes de la entrega.
- **Casos de conflicto de sincronización (U8):** C2 (cliente acepta un cambio de precio, se reenvía con UUID nuevo) y C4 (producto dado de baja, la fila queda marcada) resueltos correctamente; C6 (reenvío del mismo UUID) rechazado como se esperaba por la restricción `UNIQUE`.
- **Comparación NoSQL vs. relacional (U7):** integridad referencial confirmada como ventaja de PostgreSQL (MongoDB acepta una referencia a un producto inexistente, PostgreSQL la rechaza con un trigger real); la supuesta ventaja de velocidad de lectura de MongoDB por evitar *joins* **no se confirmó** a la escala probada (ambos motores respondieron en ~1-2 ms).

4. Qué decisiones de diseño se tomaron

- **U4:** se optó por mantener la tabla legacy sin eliminar durante la migración (en vez de un `DROP` inmediato), precisamente para que el plan de reversión no dependa de reconstruir datos ya borrados.
- **U5:** se rechazó la propuesta de la IA de otorgar `GRANT ALL` a los roles de solo lectura (`rol_reportes`, `rol_soporte`) “para cubrir necesidades futuras”, por violar directamente el principio de mínimo privilegio que es el objetivo central de la unidad.
- **U6:** el code review de U8 encontró que restaurar solo la tabla `producto` (en vez del respaldo completo) rompe la vista `v_snapshot_movil` por la FK a `categoria` — se confirmó la decisión ya tomada en U6 de respaldar la base completa, no tablas sueltas.
- **U7:** se decidió mantener PostgreSQL como fuente de verdad y usar MongoDB como una réplica documental de solo lectura para consultas puntuales, en vez de reemplazar el modelo relacional, en base a los resultados medidos (no a la intuición de que “NoSQL es más rápido”).
- **U8:** se mantuvo la regla de “el móvil propone, el servidor dispone” (`SIMULADO_APROBADO` nunca es una venta real) en todos los casos de conflicto, y se descartó explícitamente un trigger que descontara `stock_visto` desde la tablet, por romper esa regla.

5. Qué herramientas de IA se usaron

Se usó **OpenCode** (con GitHub Models como proveedor) y **Claude** a lo largo de las cinco unidades, con specs de Kiro como entrada estructurada en U3, U5, U6 y U8. El detalle completo de prompts, qué se aceptó y qué se rechazó está en `BITACORA.md` (consolidada para U4 a U8) y en las declaraciones de uso de IA de cada unidad: `duia.md` (unidades anteriores), `docs/DUIA_TP8.md` (U8), y `informe_u7.pdf` del repositorio `nosql-mongo` (U7). El caso más significativo de corrección a una propuesta de IA está documentado en `BITACORA.md`, sección

“Caso más significativo de error de la IA”.

6. Qué no se entrega

Siguiendo el mismo criterio que en U6: no se incluye ninguna contraseña real, los datos de prueba son inventados o anonimizados (`usuario_anon`, `cliente_feria_xx`, emails `@foodstore.test`), y el `.dump` de respaldo incluido es de un volumen de prueba, no de datos reales de producción.